

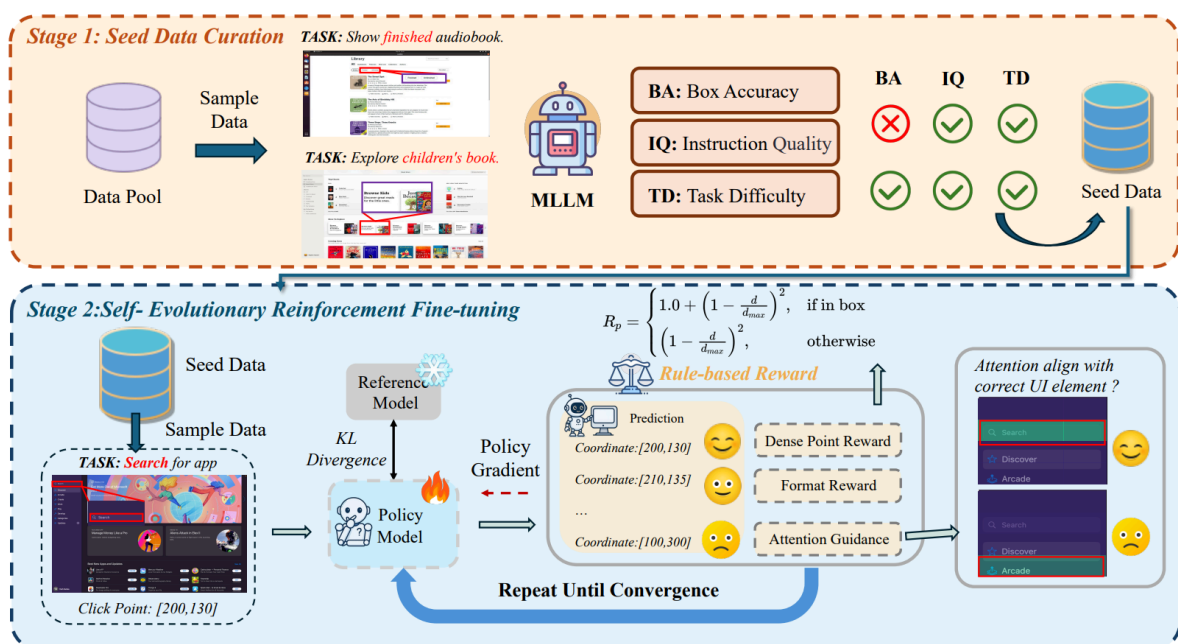
Grounding

0. Enhancing Visual Grounding for GUI Agents via Self-Evolutionary Reinforcement Learning

Grounding

给出三种 Grounding 的解决方案:

1. 预先通过 MLLM 筛选数据
2. 设计连续的奖励函数, 并通过 GRPO 强化对 bounding box 的感知
3. 通过观察 LLM attention 层的数值, 把 attention 层映射到原图上, 来把失败的损失值忽略掉



1. data curation

分为 Instruction quality, Bounding box accuracy, Task difficulty 来评判数据质量

2. GRPO

使用如下奖励函数表示 Grounding 的准确度:

$$d = \sqrt{\left(\frac{x}{W} - \frac{x_1 + x_2}{2 \cdot W}\right)^2 + \left(\frac{y}{H} - \frac{y_1 + y_2}{2 \cdot H}\right)^2},$$

$$R_p = \begin{cases} 1.0 + \left(1 - \frac{d}{d_{max}}\right)^2, & \text{if } \begin{cases} x_1 \leq x \leq x_2, \\ y_1 \leq y \leq y_2 \end{cases} \\ \left(1 - \frac{d}{d_{max}}\right)^2, & \text{otherwise} \end{cases}$$

3. attention

将原本的注意力层的第 i, j 个权重记为 $attn[i, j]$

那么使用如下 binary 函数来判定损失是否有效:

$$f(attn, gt^{bbox}, \tau) = \begin{cases} 1, & \text{if } P_{\text{peak}} \wedge P_{\text{global}} \\ 0, & \text{otherwise} \end{cases}$$

其中的两个 P :

$$P_{\text{global}} = 1 \left(\frac{1}{H_{\text{gt}} W_{\text{gt}}} \sum_{i=x_1}^{x_2} \sum_{j=y_1}^{y_2} attn[i, j] > \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} attn[i, j] \right),$$

P_{global} 表示 bounding box 内的注意力权重平均是否高于全局平均, 如果是, 则有效地注意到了 bounding box 内的内容

$$P_{\text{peak}} = 1 \left(\max_{(i,j) \in [x_1, x_2] \times [y_1, y_2]} attn[i, j] > \tau \right),$$

P_{peak} 表示 bounding box 内是否有某一权重大于阈值 τ , 说明注意到峰值

最后使用过滤函数 f 进行 GRPO:

$$\mathcal{L}(\theta) = f(attn, gt^{bbox}, \tau) \times \mathbb{E}_t \left[-\frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} A_t + \gamma \cdot \text{KL}[\pi_{\text{old}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right],$$

Memory

1. A-Mem: Agentic Memory for LLM Agents

Memory

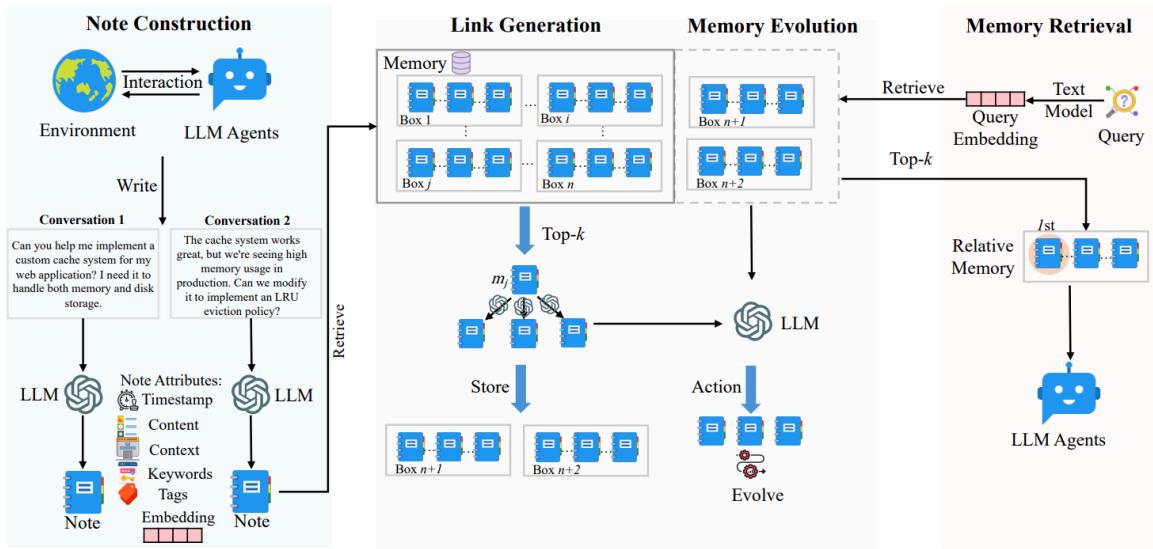
使用类似 RAG 的方法, 把历史互动 c_i 用大模型生成一些相关的总结, 并把这些文字通过 encoder 生成一个向量 e_i

计算 cosine similarity 判断哪个记忆与现在相关, 把这些记忆与现在记忆生成一个 link

之后对于 link 到的记忆进行更新

在 LLM Agents 行动时查询 Memory 来更好地理解任务

对于 GUI Agent, 既可以在一次任务过程中的每个决策时调用 A-MEM 的更新记忆 / 获取记忆功能; 同时在不同的任务之间也能长期记忆, 回忆起相似任务的行动细节



1. Node Construction

一条记忆表示为

$$m_i = \{ c_i, t_i, K_i, G_i, X_i, e_i, L_i \}$$

其中 c_i 是原始互动内容

L_i 是与其他语义相似的记忆的链接

ParseError: KaTeX parse error: Can't use function '\$' in math mode at position 44: ...i||t_i||P_{s1}}\}\$
 $e_i = f_{en...}$

2. Link Generation

语义 cosine similarity topk 相似的记为 $\mathcal{M}_{near}^n$

$$L_i = LLM(m_n || \mathcal{M}_{near}^n || P_{s2})$$

3. Memory Evolution

对于 $\mathcal{M}_{near}^n$ 中的每个 m_j^* , 让 LLM 决定是否对其进行 update neighbor 或者 strengthen (更新 neighbor 的内容, 或者加入新的 link)

4. Memory Retrieval

选择 cosine similarity topk 的记忆给 LLM Agents

Task decomposition

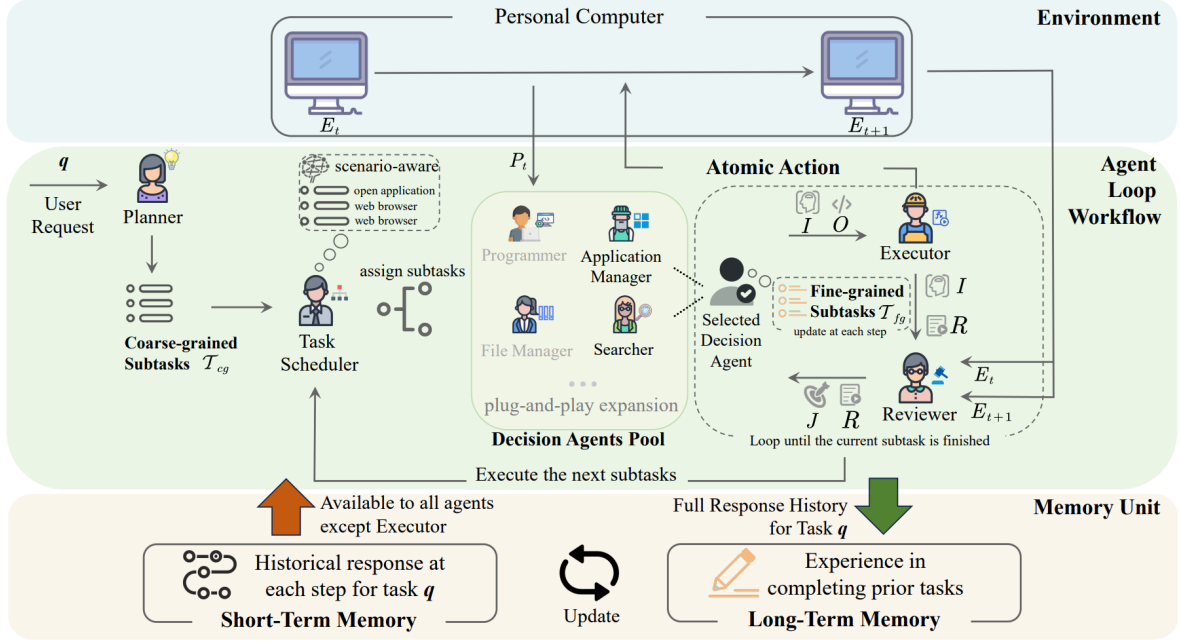
2. COLA: A SCALABLE MULTI-AGENT FRAMEWORK FOR WINDOWS UI TASK AUTOMATION

Task decomposition, (Memory)

类似 MoE, 先把用户要求输入给 Planner 产生粗粒度的子任务, 之后 Task Scheduler 把每个子任务分配给最优的 Decision Agent

对于每个 Decision Agent 循环进行细粒度地拆分并进行原子化操作, 让 Reviewer 根据环境评判任务完成度, 直到整个子任务完成才停止循环

同时记忆模块与 A-MEM 类似, 只不过区分长短期记忆



1. Planner

$$\mathcal{T}_{cg} = \{s_1, \dots, s_k\} = PL(q, LT_t^n, ST_t^m)$$

PL 为 Planner, 通过用户询问 q , 长期记忆前 n 相关与短期记忆前 m 最近使用地记忆作为 prompt, 来产生 subtask

2. Task scheduler

$$\mathcal{D} = \{(role_1, rt_1), \dots, (role_k, rt_k)\} = TS(\mathcal{T}_{cg}, DA_{desc}, LT_t^n, ST_t^m)$$

对于每个 Agent 给一个描述, 组成 DA_{desc} , 之后结合上一步, 输出每个 Agent $role_i$ 和对应的子任务

3. Decision Agent Pool

$$(I, O, \mathcal{T}_{fg}) = DA_{role_k}(q, rt_k, P_t, J, LT_t^n, ST_t^m)$$

P_t 是通过 visual backbone 识别环境 E_t 得到 (对 PC 环境进行 OCR), J 为 Reviewer 对 Agent 的行为做出的判断

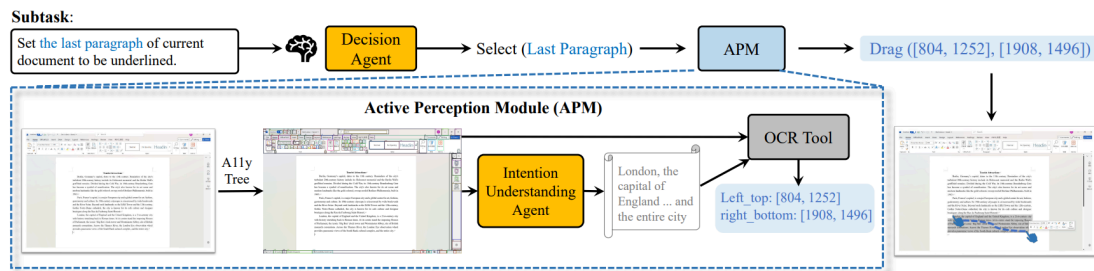
O 是输出的原子化操作, I 是对应的意图

之后每一步都更新 $\mathcal{T}_{fg}$, 直到所有 $\mathcal{T}_{fg}$ 都完成, 这个 Agent 的子任务完成, task scheduler 分配进行下一个子任务

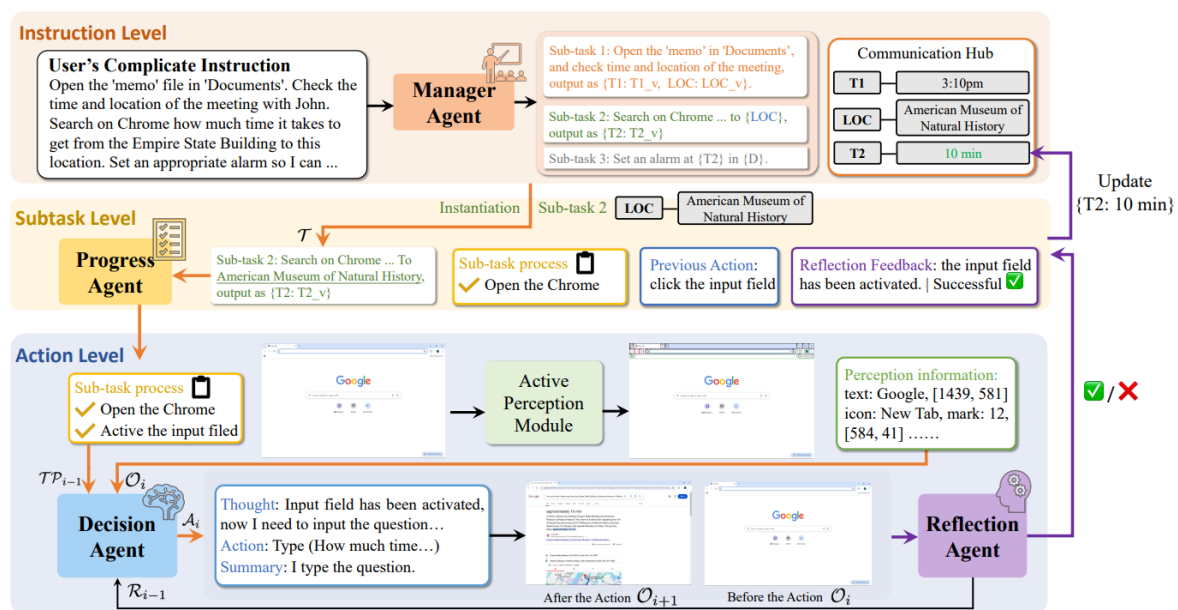
3. PC-Agent: A Hierarchical Multi-Agent Collaboration Framework for Complex Task Automation on PC

Task decomposition, (Grounding)

使用 Active Perception Module 来增强 Grounding 等与环境互动的部分



同时将任务拆分成三层: 最上层先将指令拆分, 中间层由 Progress Agent 管理子任务的进度, 最底层由使用 APM 增强的 MLLM Agent 完成任务, 并由另一个 MLLM Agent 进行评估



1. Active Perception Module (APM for Grounding)

- Interactive element perception

使用 pywinauto API 来获取元素坐标与描述

- Text perception

使用 MLLM-driven 的意图理解 Agent 和 OCR 来确定文本的范围

2. Hierarchical Multi Agent

1. Instruction level

使用一个 Manager Agent 将 instruction 分解成 subtasks

同时管理子任务之间的关系

一共四种, 完全独立, 与前面子任务有关, 给后面子任务提供信息, 以及两者都有

2. Subtask level

根据 decision agent 和 reflection agent 总结子任务的进度

3. Action level

decision agent 根据当前任务进度, 以及要求来行动

reflection agent 根据 decision agent 的行为意图以及结果判断是否成功

4. ASSISTGUI: Task-Oriented PC Graphical User Interface Automation

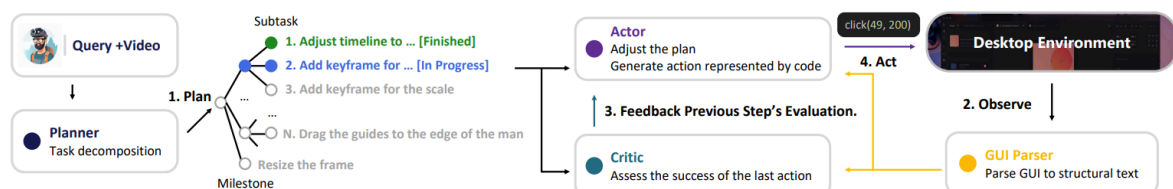
Task Decomposition

与 PC-Agent 类似

先用一个 Planner 进行基于 query 的任务拆分 (可以接受一个 video)

再使用一个 GUI Parser 把 PC element 解析出来

最后由 Actor 决定行动, 并由 Critic 进行评估反馈给 Actor



总结

上面三种方法都是先使用 Planner 将用户需求分解成粗粒度的子任务, 之后分配给多个 expert 来执行 (可由统一的 agent 分配并管理); 最后 expert 分成 actor 和 critic, 一个 agent 负责根据回报与当前状态产生行动, 另一个 agent 对于行动与环境变化产生回报

5. Advancing Agentic Systems: Dynamic Task Decomposition, Tool Integration and Evaluation using Novel Metrics and Dataset

Task Decomposition

任务不在分割成一条链, 而是用有向无环图表示, 这样允许 task 并行处理, 同时可以使用 Graph-enhanced LLM 来当 Agent

其他部分与上述大框架基本一致

- 主要的问题:

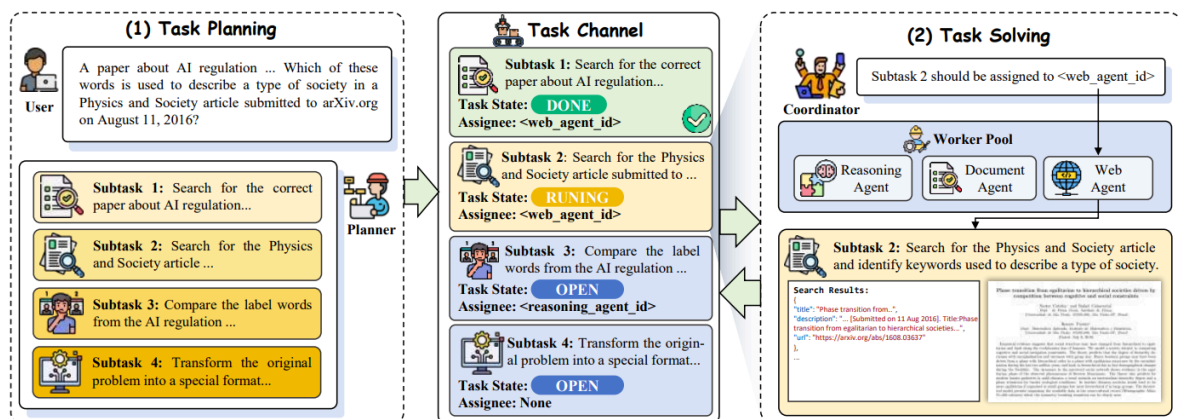
1. 使用 DAG 比链式的子任务结构似乎只有并行运行有提升, 对于具体的任务分割没帮助

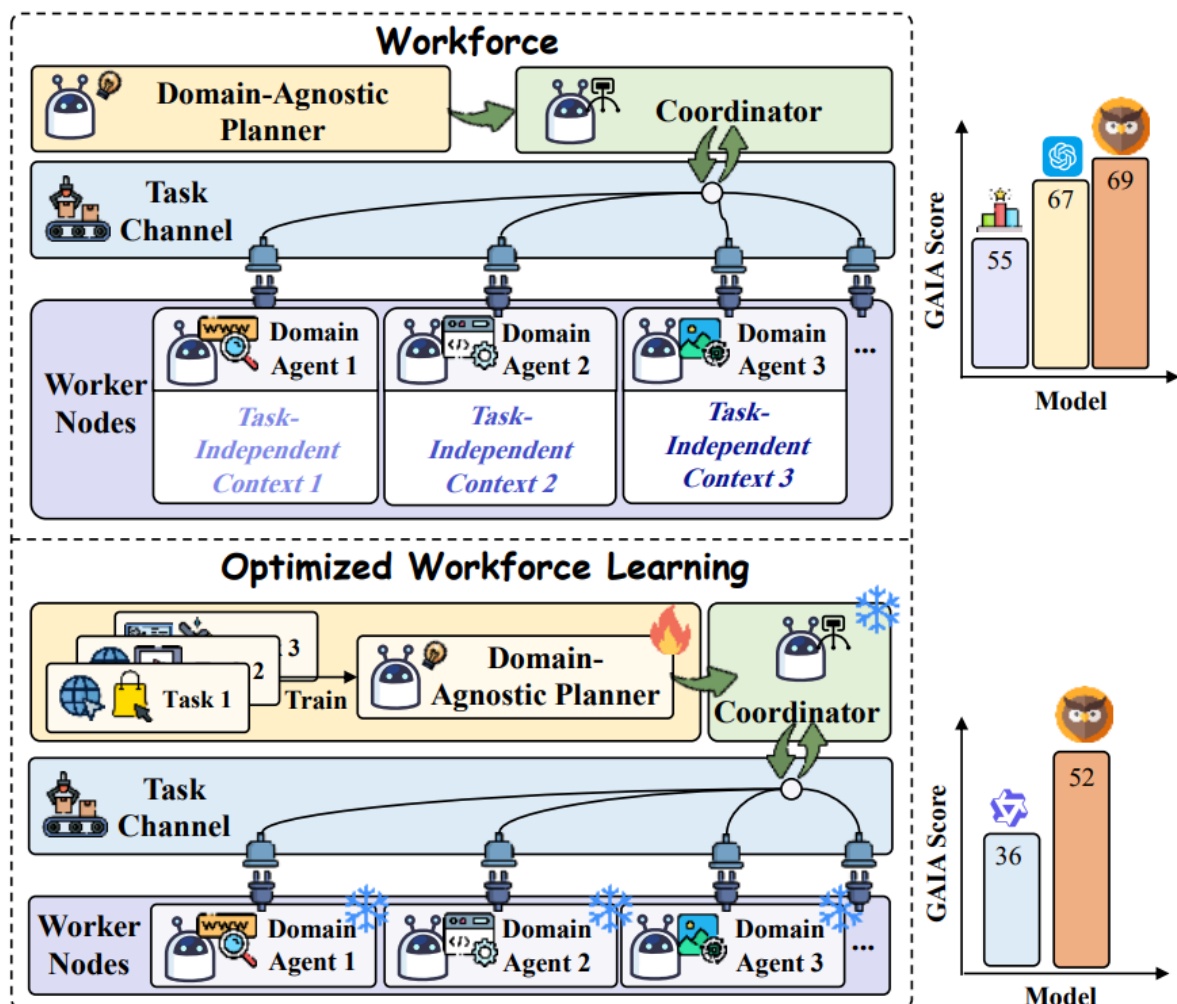
6. OWL: Optimized Workforce Learning for General Multi-Agent Assistance in Real-World Task Automation

Dynamic Task Decomposition

同样先进行 Task Planning, 交给 Coordinator 进行任务分工, 每个 Worker Nodes 中的 Agent 根据独立上下文完成子任务

同时任务结果返回给 Task Channel, Coordinator 从中获取任务结果并总结发给 Planner





1. Replanning Mechanism

如果子任务全部成功, Planner 总结并结束总任务; 如果子任务失败, Planner 重新规划 subtasks

2. Optimized Workforce Learning (OWL)

同时对于 Planner, 可以进一步通过训练加强表现

先使用 GPT-4o-mini 作为底座, 应用 Workforce 进行整体运行 trajectory 的记录, 通过不同数据集的不同指标筛选高质量 trajectory, 进行 SFT

再使用 SFT 后的 LLM 生成 n 个不同的轨迹, 标记一条偏好的轨迹, 使用 DPO 学习这个偏好

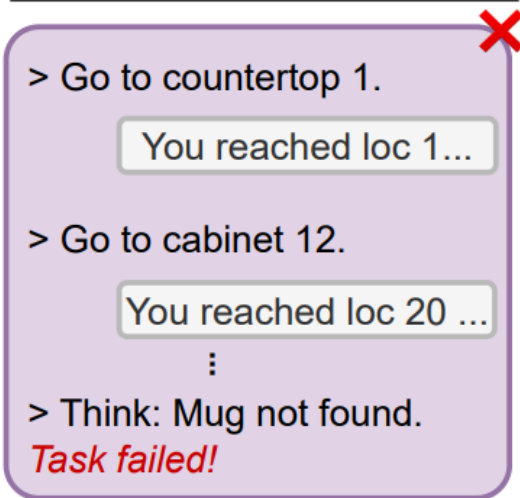
7. ADAPT: As-Needed Decomposition and Planning with Language Models

Dynamic Task Decomposition

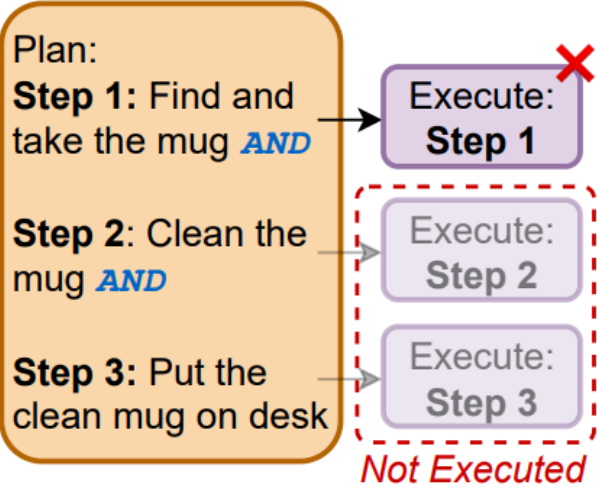
与其他静态方法对比, ADAPT 可以动态调整 plan; 同时 plan 之间不是简单的顺序完成, 而是允许条件判断

Task: Put a clean mug on desk.

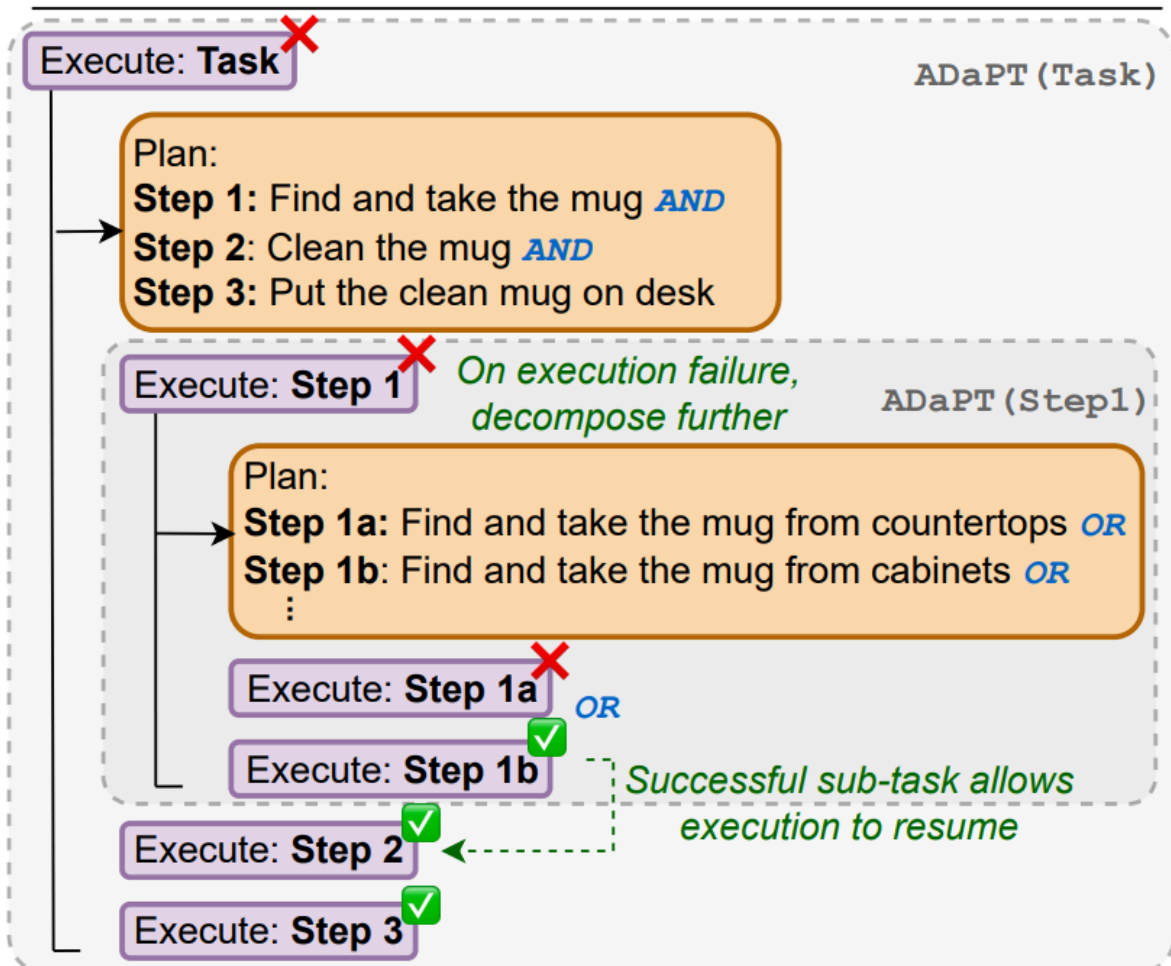
Iterative Executor (ReAct)

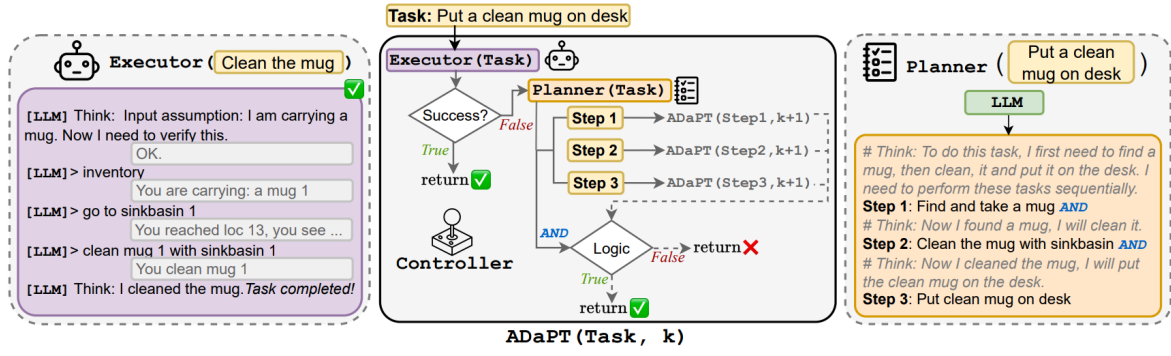


Plan-and-Execute



ADaPT (Recursive Decomposition, As-needed)





1. Planner

使用 LLM 作为 Planner, 生成粗粒度的子任务以及任务之间的逻辑运算符, 如子任务之间的关系是 AND 还是 OR

2. ADaPT

把 Planner 规划出的子任务交给 Executor 执行, 如果失败就进一步细分, 直到任务完成或达到最大深度

Algorithm 1 Algorithm for ADAPT

```

1: function ADAPT(Task  $T$ , Current depth  $k$ )
2:   // ADAPT( $\cdot$ ) Generates success heuristic value
   // completed for the task  $T$ . Initialized with  $k = 1$ .
3:   // Base case: terminate on reaching maximum depth
4:   if  $k > d_{\max}$  then return False
5:   // Execute the task/sub-task to assess if the LLM can
   // directly perform it using LLM-generated success.
6:    $completed \leftarrow \text{executor}_{\text{LLM}}(T)$ 
7:   // Plan only when the executor fails.
8:   if  $completed$  is False then
9:     // Using the LLM, decompose the task into a set
     // of sub-tasks,  $\mathcal{P}$ , and a Boolean function,  $logic(\cdot)$ ,
     // that combines output of the sub-tasks.
10:     $\mathcal{P}, logic \leftarrow \text{planner}_{\text{LLM}}(T)$ 
11:    // Get the outputs for individual sub tasks
12:     $\mathcal{O} = \{\text{ADAPT}(T_{\text{sub}}, k+1) | T_{\text{sub}} \in \mathcal{P}\}$ 
13:    // Combine the outputs of the sub tasks
14:     $completed \leftarrow logic(\mathcal{O})$ 
15:  return  $completed$ 

```

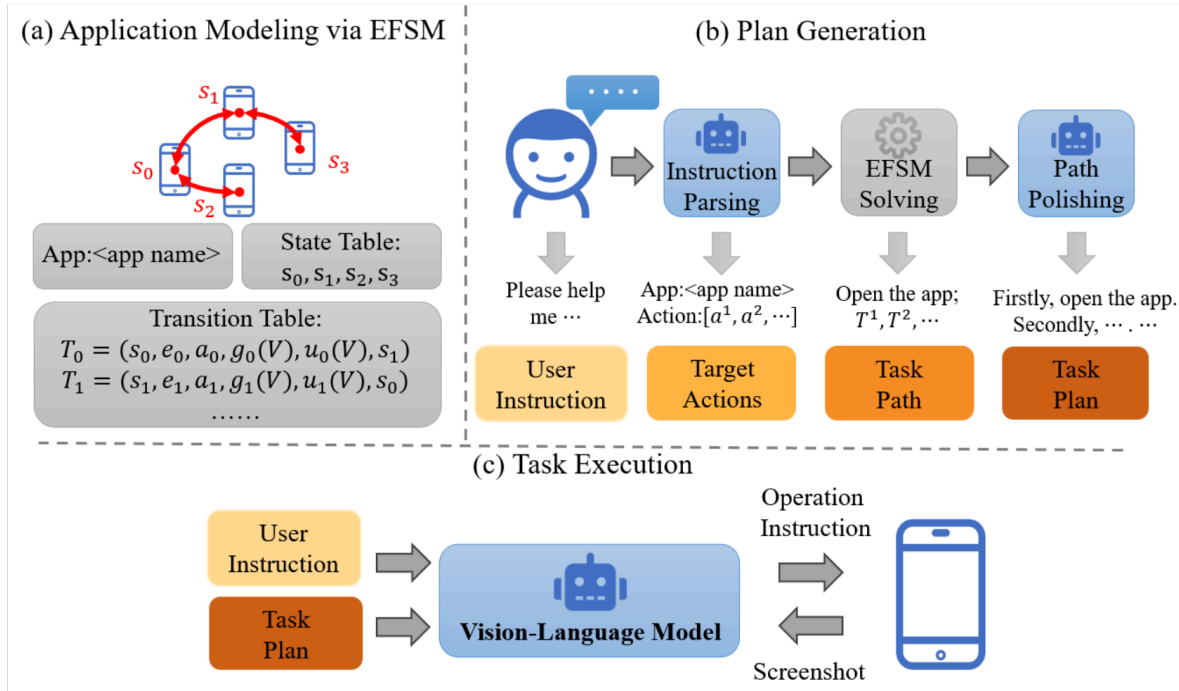
Others

8. Building a Stable Planner: An Extended Finite State Machine Based Planning Module for Mobile GUI Agent

Planning, MultiApp

这篇讲的是多应用的场景下, 可以先对每个应用手动写出有限状态机, 之后对每个 app 分解出对应的有限状态机 ϵ_j 和目标行为 A_j , 之后用 BFS 找出每个 ϵ_j 的最短路径解

之后用 LLM 把路径改写成自然语言的计划, 用 MLLM 执行计划



- 主要的问题:

1. 第一步 Application Modeling vis EFSM 是人工的, 是否可以换成 LLM 自动探索
2. 对于复杂应用, 如包含搜索的应用, 有限状态机是否会过于复杂

Symbol	Explanation	Examples
$s \in S$	A screen or page of the application	Camera home page; Camera settings page
$e \in E$	A sequence of user operations performed on the GUI	Click the button at the bottom of the screen, then click again after {duration}
$a \in A$	A primary function performed by the application	Take a photo; Record a video of {duration}
V	Internal variables describing the application's configuration	Video mode = True; Front camera mode = False
$g(V)$	Guard conditions that must be satisfied for a transition to occur	if Video mode = True
$u(V)$	Update function applied to variables during a transition	Video mode $\rightarrow$ False

Algorithm 1 Workflow of SPlanner

Application Modeling: Use EFSM to model all target applications and obtain the EFSM set $\mathcal{F}$ as defined in Eq. 2.

Plan Generation: Given user instruction I and EFSM set $\mathcal{F}$,

1. Use an LLM to parse I , producing $[\varepsilon_1, \varepsilon_2, \dots, \varepsilon_j]$ and $[A_1^T, A_2^T, \dots, A_j^T]$, as shown in Eq. 5.
2. Use a BFS-based solver to compute the execution paths $p_i = BFS(\varepsilon_i, A_i^T)$ for each app, and aggregate them into the global path plan $P = (p_1, p_2, \dots, p_j)$, as shown in Eq. 6.
3. Use an LLM to combine I and P to generate the final natural language plan.

Task Execution: Given initial GUI screenshot S_0 , instruction I , generated plan $Plan$, initial action history $H_0 = \emptyset$, and step counter $i = 1$,

while task is not completed **and** step limit not reached **do**

 Generate the next operation instruction $O_i = VLM(I, S_i, Plan, H_i)$.

 Update history: $H_{i+1} = H_i + O_i$.

 Increment step: $i \leftarrow i + 1$.

end while
